# JHU AI Engineering

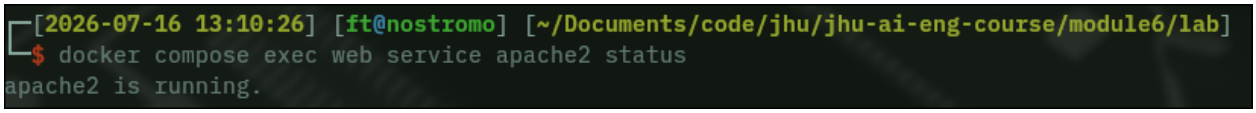
## Module 6 - SQL Server Security

Fred Teumer

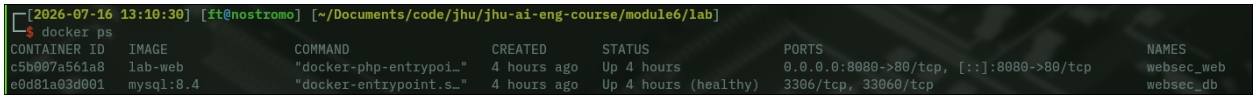
# Part 1: LAMP Stack and Database Setup Deliverables

1. **Status of `apache2`**

   *Note: I chose to run this locally via `docker compose`. This doesn't run as a `systemd` service and instead runs as a native binary within the container (Additional screenshot below).*

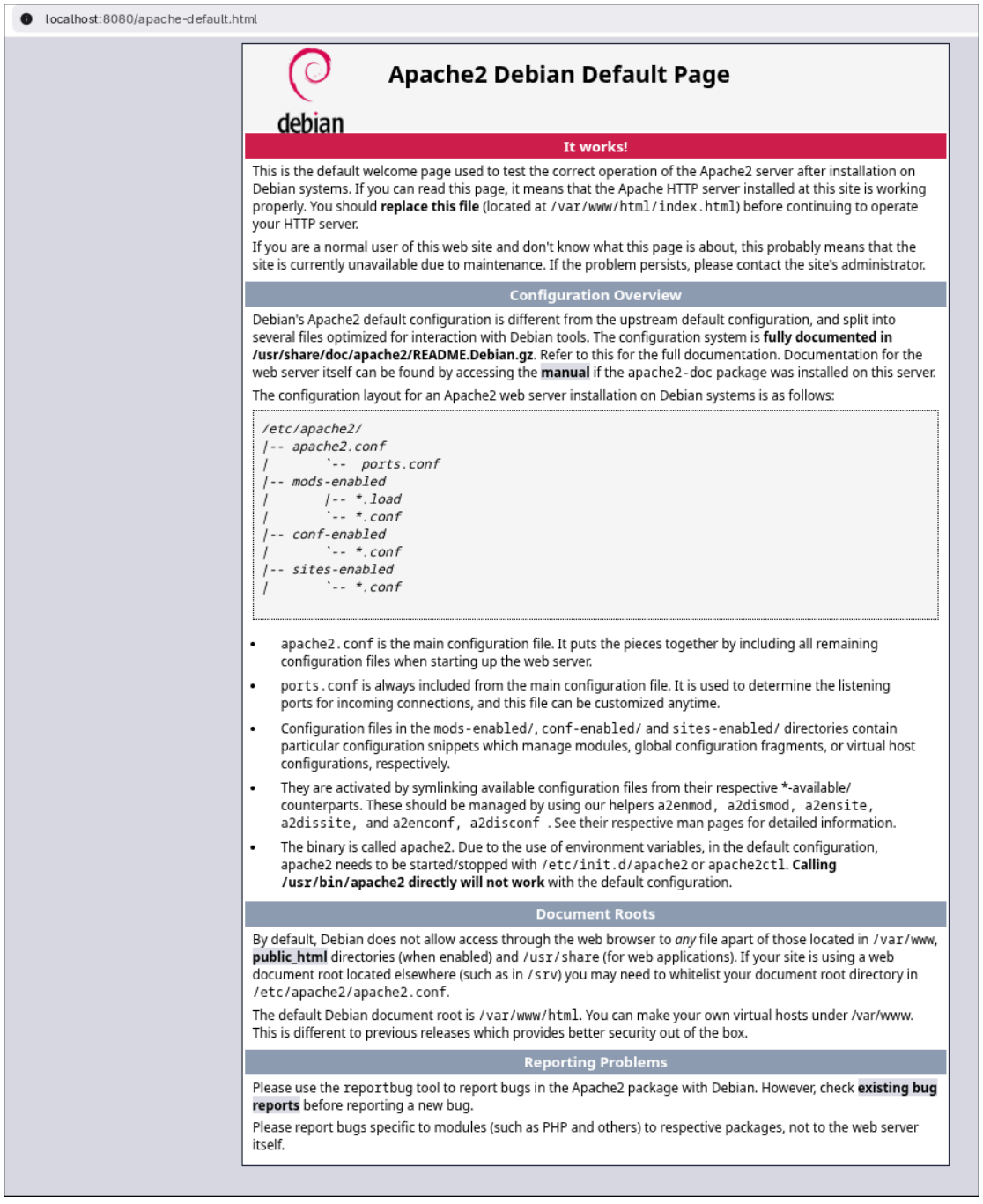



*A screenshot of a docker command demonstrating the status of the `apache2` service [14]*




*A screenshot of docker ps demonstrating a healthy running `apache2` web server*

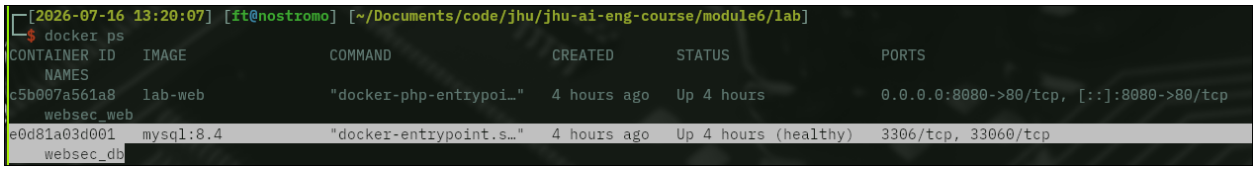
2. **Your apache2 default page:**
   http://localhost:8080/apache-default.html




*A screenshot of the Apache2 Default page*

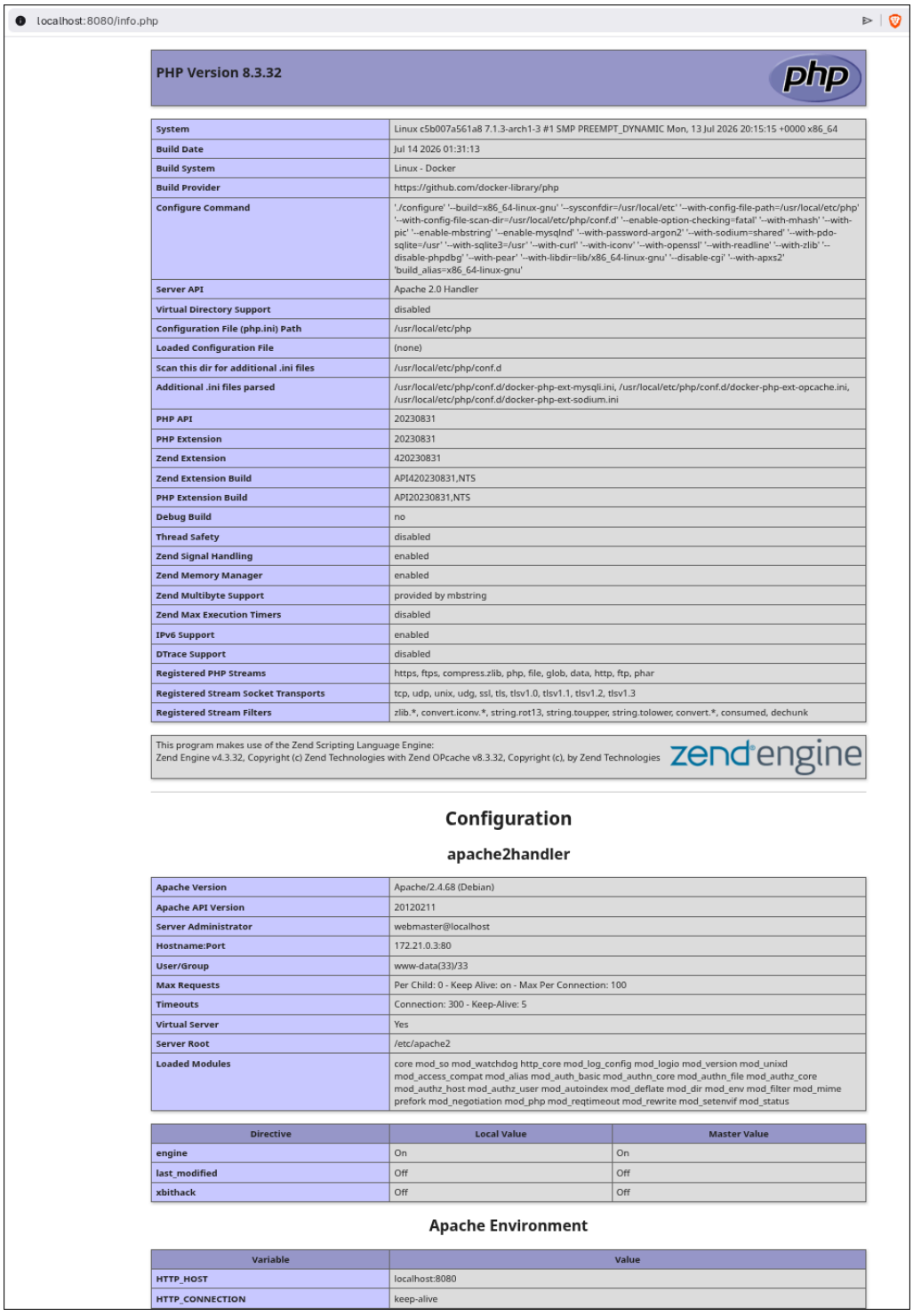
3. ## Status of MySQL


*A screenshot of the `mysqladmin version` command, showing a working database [10] [11]*




*A screenshot of `docker ps` showing a healthy and running database*

## 4. PHP info page:



localhost:8080/info.php

**PHP Version 8.3.32**

| | |
|---|---|
| **System** | Linux c5b007a561a8 7.1.3-arch1-3 #1 SMP PREEMPT_DYNAMIC Mon, 13 Jul 2026 20:15:15 +0000 x86_64 |
| **Build Date** | Jul 14 2026 01:31:13 |
| **Build System** | Linux - Docker |
| **Build Provider** | https://github.com/docker-library/php |
| **Configure Command** | './configure' '--build=x86_64-linux-gnu' '--sysconfdir=/usr/local/etc' '--with-config-file-path=/usr/local/etc/php' '--with-config-file-scan-dir=/usr/local/etc/php/conf.d' '--enable-option-checking=fatal' '--with-mhash' '--with-pic' '--enable-mbstring' '--enable-mysqlnd' '--with-password-argon2' '--with-sodium=shared' '--with-pdo-sqlite=/usr' '--with-sqlite3=/usr' '--with-curl' '--with-iconv' '--with-openssl' '--with-readline' '--with-zlib' '--disable-phpdbg' '--with-pear' '--with-libdir=lib/x86_64-linux-gnu' '--disable-cgi' '--with-apxs2' 'build_alias=x86_64-linux-gnu' |
| **Server API** | Apache 2.0 Handler |
| **Virtual Directory Support** | disabled |
| **Configuration File (php.ini) Path** | /usr/local/etc/php |
| **Loaded Configuration File** | (none) |
| **Scan this dir for additional .ini files** | /usr/local/etc/php/conf.d |
| **Additional .ini files parsed** | /usr/local/etc/php/conf.d/docker-php-ext-mysqli.ini, /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini, /usr/local/etc/php/conf.d/docker-php-ext-sodium.ini |
| **PHP API** | 20230831 |
| **PHP Extension** | 20230831 |
| **Zend Extension** | 420230831 |
| **Zend Extension Build** | API420230831,NTS |
| **PHP Extension Build** | API20230831,NTS |
| **Debug Build** | no |
| **Thread Safety** | disabled |
| **Zend Signal Handling** | enabled |
| **Zend Memory Manager** | enabled |
| **Zend Multibyte Support** | provided by mbstring |
| **Zend Max Execution Timers** | disabled |
| **IPv6 Support** | enabled |
| **DTrace Support** | disabled |
| **Registered PHP Streams** | https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar |
| **Registered Stream Socket Transports** | tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3 |
| **Registered Stream Filters** | zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk |

This program makes use of the Zend Scripting Language Engine:
Zend Engine v4.3.32, Copyright (c) Zend Technologies with Zend OPcache v8.3.32, Copyright (c), by Zend Technologies

**Configuration**

**apache2handler**

| | |
|---|---|
| **Apache Version** | Apache/2.4.68 (Debian) |
| **Apache API Version** | 20120211 |
| **Server Administrator** | webmaster@localhost |
| **Hostname:Port** | 172.21.0.3:80 |
| **User/Group** | www-data(33)/33 |
| **Max Requests** | Per Child: 0 - Keep Alive: on - Max Per Connection: 100 |
| **Timeouts** | Connection: 300 - Keep-Alive: 5 |
| **Virtual Server** | Yes |
| **Server Root** | /etc/apache2 |
| **Loaded Modules** | core mod_so mod_watchdog http_core mod_log_config mod_logio mod_version mod_unixd mod_access_compat mod_alias mod_auth_basic mod_authn_core mod_authn_file mod_authz_core mod_authz_host mod_authz_user mod_autoindex mod_deflate mod_dir mod_env mod_filter mod_mime prefork mod_negotiation mod_php mod_reqtimeout mod_rewrite mod_setenvif mod_status |

| Directive | Local Value | Master Value |
|---|---|---|
| **engine** | On | On |
| **last_modified** | Off | Off |
| **xbithack** | Off | Off |

**Apache Environment**

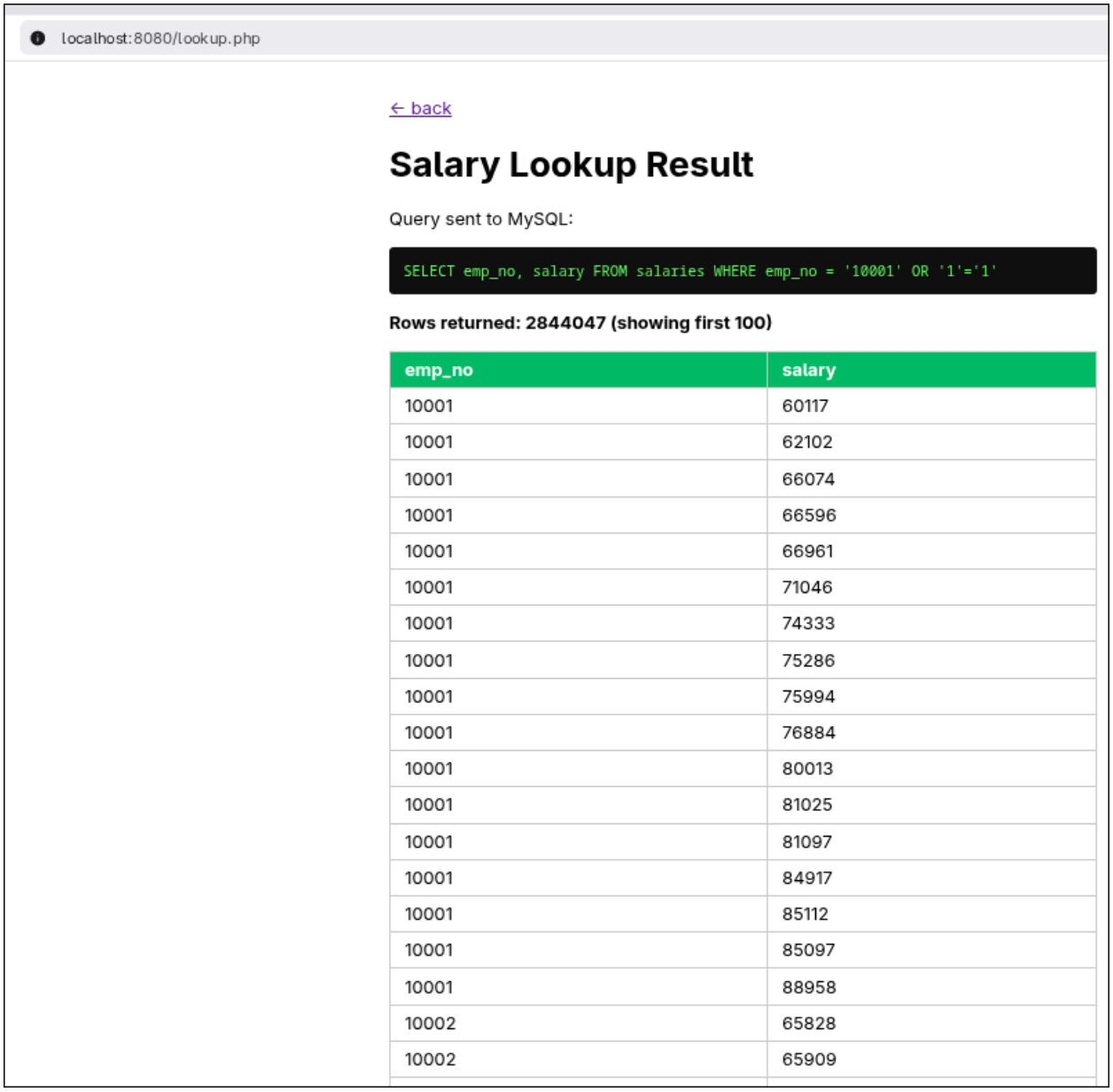
| Variable | Value |
|---|---|
| **HTTP_HOST** | localhost:8080 |
| **HTTP_CONNECTION** | keep-alive |

*A Subsection of the PHP default info page*

# Part 2: SQL Injection Deliverables

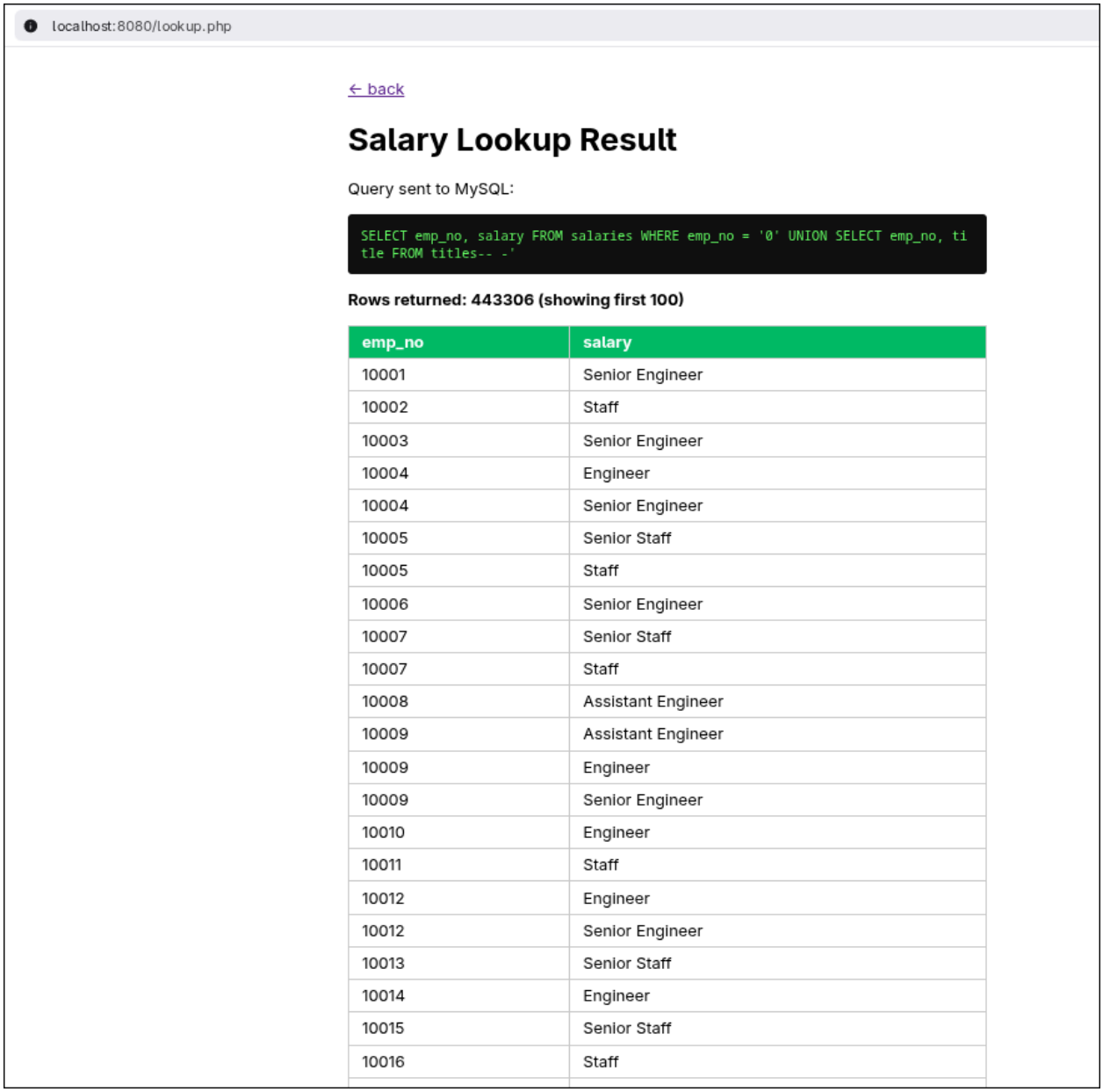
## A. Provide the annotated screen snips of your 3 attempts to conduct SQL Injections.

### 1. Attempt 1: Tautology (boolean bypass)



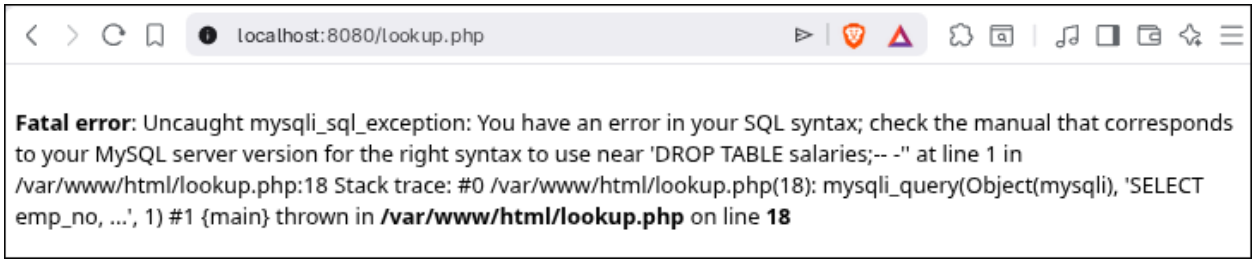

*Using SQL injection to retrieve the entire table [7][8]*

## 2. Attempt 2: `UNION`-based (cross-table exfiltration)




*Using SQL injection to pull from other tables in the DB. [9]*

3. **Attempt 3: Stacked query (destructive)**
   Injected value: `10001'; DROP TABLE salaries;-- -`




*A failed SQL injection attempting to drop a table.*

## B. For each of the SQLi queries you selected, briefly explain:

### i. Why did you choose this query?

**Attempt 1: Tautology (boolean bypass)**
This is the classic authentication/filter bypass. My `'` closes the intended string, and `OR '1'='1'` appends a condition that's always true.

**Attempt 2: UNION-based (cross-table exfiltration)**
I chose this one to prove I can pull data from a *different* table (titles) through the salary form. The real danger of injection, exfiltrating arbitrary data.

**Attempt 3: Stacked query (destructive)**
I chose the most damaging payload: stack a second statement (`DROP TABLE`) after the query to destroy data.

### ii. What were your expected results?

**Attempt 1: Tautology (boolean bypass)**
Return rows I shouldn't have access to (more than one employee).

**Attempt 2: UNION-based (cross-table exfiltration)**
Job-title data appearing where salaries should be.

**Attempt 3: Stacked query (destructive)**
The `salaries` table gets deleted.

### iii. What was the actual result?

**Attempt 1: Tautology (boolean bypass)**
✅ Succeeded. All 2,844,047 salary rows returned instead of one employee's 17.

**Attempt 2: UNION-based (cross-table exfiltration)**
✅ Succeeded. 443,306 rows from titles surfaced in the salary column.

**Attempt 3: Stacked query (destructive)**
❌ Failed. `mysqli_query()` executes only one statement [1], so the `;`-separated `DROP` is rejected as a syntax error and never executes.

## Part 3: Security Analysis Deliverables

Contrary to what I expected going into this assignment, I was able to perform a successful SQL injection attack: two of my three payloads worked against the vulnerable `lookup.php` page.

The tautology `10001' OR '1'='1` returned all 2,844,047 rows of the salaries table instead of a single employee's record, and the `UNION`-based payload exfiltrated 443,306 rows from the unrelated titles table straight through the salary lookup form. The root cause was never the database engine or the language version; it was that my PHP concatenated untrusted user input directly into the query string, so the input was free to alter the query's structure.

What did stop me was narrower than I assumed. My third payload, the stacked 10001'; DROP TABLE salaries;, failed because the code used mysqli_query(), which executes only a single statement per call and rejected the second, semicolon-separated command as a syntax error. The DROP never ran, and the table remained fully intact. This is an important distinction that my research into MySQLi and PHP 8 clarified: MySQLi did not sanitize my input or make my code "safe," and PHP 8 introduced no protection against injection itself (its main relevant change was making mysqli throw exceptions on error by default, which is why the failed attack surfaced as an uncaught mysqli_sql_exception [3][4]). The single-statement behavior of mysqli_query() blocked one category of attack – piggybacked queries – while leaving classic tautology and UNION attacks wide open. An attacker who wanted stacked queries could simply have called mysqli_multi_query() instead [2].

The real, durable defense is to stop mixing code and data altogether [6]. When I reimplemented the same lookup with a prepared statement and a bound integer parameter [5] in `secure_lookup.php`, the identical `' OR '1'='1` payload that had dumped the entire table was treated as literal data, cast to the integer 10001, and returned only that one employee's seventeen rows.

My takeaway from this assignment is that library and version defaults offer, at best, partial and incidental protection, and that parameterized queries – not MySQLi, escaping, or PHP 8 – are what actually neutralize SQL injection.

# References


1. PHP Documentation. "mysqli::query / mysqli_query." The PHP Group. https://www.php.net/manual/en/mysqli.query.php (accessed July 16, 2026).
2. PHP Documentation. "mysqli::multi_query / mysqli_multi_query." The PHP Group. https://www.php.net/manual/en/mysqli.multi-query.php (accessed July 16, 2026).
3. PHP Documentation. "Backward Incompatible Changes — PHP 8.0.x to 8.1.x (MySQLi)." The PHP Group. https://www.php.net/manual/en/migration81.incompatible.php (accessed July 16, 2026).
4. PHP Documentation. "mysqli_driver::$report_mode (MySQLi reporting mode)." The PHP Group. https://www.php.net/manual/en/mysqli-driver.report-mode.php (accessed July 16, 2026).
5. PHP Documentation. "MySQLi — Prepared Statements." The PHP Group. https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php (accessed July 16, 2026).
6. OWASP. "SQL Injection Prevention Cheat Sheet." OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (accessed July 16, 2026).
7. OWASP. "SQL Injection." OWASP Community. https://owasp.org/www-community/attacks/SQL_Injection (accessed July 16, 2026).
8. PortSwigger. "SQL injection." Web Security Academy. https://portswigger.net/web-security/sql-injection (accessed July 16, 2026).
9. PortSwigger. "SQL injection UNION attacks." Web Security Academy. https://portswigger.net/web-security/sql-injection/union-attacks (accessed July 16, 2026).
10. Giuseppe Maxia (datacharmer). "test_db — MySQL Employees Sample Database." GitHub. https://github.com/datacharmer/test_db (accessed July 16, 2026).
11. Oracle. "Employees Sample Database." MySQL Documentation. https://dev.mysql.com/doc/employee/en/ (accessed July 16, 2026).
12. w3schools. "PHP MySQL Select Data." https://www.w3schools.com/php/php_mysql_select.asp (accessed July 16, 2026).
13. w3schools. "PHP Form Handling." https://www.w3schools.com/php/php_forms.asp (accessed July 16, 2026).
14. Docker. "php — Official Image (Apache variant)." Docker Hub. https://hub.docker.com/_/php (accessed July 16, 2026).